Deadlock Detector 사용자 매뉴얼

버전: 1.0.0 작성일: 2026-01-25 대상 독자: 시스템 관리자, 개발자, 운영 담당자

목차

1. 설치 및 설정
2. API 사용법
3. 모니터링 대시보드
4. 트러블슈팅

---

1. 설치 및 설정

1.1 시스템 요구사항

구성 요소	최소 버전	권장 버전	비고
Node.js	20.0.0	20 LTS	최신 LTS 권장
npm	9.0.0	10.x	Node.js와 함께 설치
MongoDB	7.0	7.0+	영구 데이터 저장
Redis	7.0	7.2+	캐싱 및 세션
메모리	512MB	2GB+	그래프 크기에 따라
디스크	100MB	1GB+	로그 및 체크포인트

1.2 의존성 서비스 설치

MongoDB 설치

```
# macOS (Homebrew)
brew tap mongodb/brew
brew install mongodb-community@7.0
brew services start mongodb-community@7.0

# Ubuntu/Debian
wget -qO - https://www.mongodb.org/static/pgp/server-7.0.asc | sudo apt-key add -
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-
org/7.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
sudo apt-get update
sudo apt-get install -y mongodb-org
sudo systemctl start mongod

# Docker
docker run -d --name mongodb -p 27017:27017 mongo:7.0
```

Redis 설치

```
# macOS (Homebrew)
brew install redis
brew services start redis

# Ubuntu/Debian
sudo apt-get install redis-server
sudo systemctl start redis-server

# Docker
docker run -d --name redis -p 6379:6379 redis:7.2
```

1.3 애플리케이션 설치

```
# 1. 프로젝트 디렉토리로 이동
cd candidates/candidate-3-deadlock-detector

# 2. 의존성 설치
npm install

# 3. TypeScript 빌드
npm run build

# 4. 환경 변수 설정
cp .env.example .env
```

1.4 환경 변수 설정

.env 파일을 편집하여 환경에 맞게 설정합니다:

```
# 서버 설정
PORT=3003
NODE_ENV=development

# MongoDB 설정
MONGODB_URI=mongodb://localhost:27017/deadlock-detector

# Redis 설정
REDIS_HOST=localhost
REDIS_PORT=6379

# 로깅 설정
LOG_LEVEL=info

# 데드락 탐지 설정
DETECTION_INTERVAL_MS=5000
MAX_CHECKPOINTS_PER_AGENT=10
```

1.5 서버 실행

```
# 개발 모드 (핫 리로드)
npm run dev

# 프로덕션 모드
npm run build
npm start

# 서버 상태 확인
curl http://localhost:3003/api/health
```

1.6 Docker 배포

```
# Dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY dist/ ./dist/
```

```dockerfile
ENV NODE_ENV=production
ENV PORT=3003

EXPOSE 3003

CMD ["node", "dist/index.js"]
```

```bash
# Docker 이미지 빌드
docker build -t deadlock-detector .

# Docker Compose 실행
docker-compose up -d
```

---

2. API 사용법

2.1 기본 URL

`http://localhost:3003/api`

2.2 에이전트 관리

에이전트 생성

```http
POST /api/agents
Content-Type: application/json

{
  "name": "Worker-Agent-1",
  "priority": 5,
  "metadata": {
    "type": "worker",
    "department": "ml"
  }
}
```

응답:

```json
{
  "success": true,
  "agent": {
    "id": "agent-abc123",
    "name": "Worker-Agent-1",
    "priority": 5,
    "status": "active",
    "heldResources": [],
    "createdAt": "2026-01-25T10:30:00.000Z"
  }
}
```

에이전트 목록 조회

```http
GET /api/agents
```

에이전트 상세 조회

```http
GET /api/agents/:agentId
```

에이전트 삭제

```http
DELETE /api/agents/:agentId
```

2.3 자원 관리

자원 생성

```
POST /api/resources
Content-Type: application/json

{
  "name": "GPU-NVIDIA-A100",
  "type": "computational",
  "instances": 1,
  "metadata": {
    "memory": "80GB",
    "location": "datacenter-1"
  }
}
```

자원 유형: | 유형 | 설명 | 예시 | |——|——|——| | `computational` | 연산 자원 | GPU, CPU | | `storage` | 저장 자원 | 디스크, DB 연결 | | `network` | 네트워크 자원 | API 슬롯, 소켓 | | `memory` | 메모리 자원 | RAM, 캐시 | | `custom` | 사용자 정의 | 기타 |

자원 요청

```
POST /api/resources/request
Content-Type: application/json

{
  "agentId": "agent-abc123",
  "resourceId": "resource-xyz789"
}
```

응답 (성공):

```
{
  "success": true,
  "message": "Resource allocated successfully",
  "allocation": {
    "agentId": "agent-abc123",
    "resourceId": "resource-xyz789",
    "allocatedAt": "2026-01-25T10:30:00.000Z"
  }
}
```

응답 (대기):

```
{
  "success": false,
  "message": "Resource not available, added to wait queue",
  "position": 3
}
```

자원 해제

```
POST /api/resources/release
Content-Type: application/json

{
  "agentId": "agent-abc123",
  "resourceId": "resource-xyz789"
}
```

2.4 데드락 탐지

데드락 감지 실행

```
POST /api/deadlock/detect
```

응답:

```
{
  "hasDeadlock": true,
  "cycles": [
    {
      "cycleId": "cycle-def456",
      "agents": ["agent-1", "agent-2", "agent-3"],
      "resources": ["resource-a", "resource-b", "resource-c"],
      "detectedAt": "2026-01-25T10:30:00.000Z"
    }
  ],
  "statistics": {
    "totalAgents": 10,
    "totalResources": 5,
    "totalEdges": 15,
    "cyclesFound": 1,
    "detectionTimeMs": 5
  }
}
```

희생자 선택

```
POST /api/deadlock/victim
Content-Type: application/json

{
  "cycleId": "cycle-def456",
  "strategy": "lowest_priority"
}
```

전략 옵션: | 전략 | 설명 | |---|---| | lowest_priority | 가장 낮은 우선순위 | | youngest | 가장 최근 생성 | | most_resources | 가장 많은 자원 보유 | | least_dependencies | 가장 적은 의존성 | | random | 무작위 선택 |

응답:

```
{
  "success": true,
  "victim": {
    "agentId": "agent-2",
    "name": "Worker-Agent-2",
    "priority": 2,
    "resources": ["resource-b"]
  },
  "strategy": "lowest_priority",
  "reason": "Agent has lowest priority (2) in the cycle"
}
```

2.5 체크포인트 및 복구

체크포인트 생성

```
POST /api/recovery/checkpoint/:agentId
```

응답:

```
{
  "success": true,
  "checkpoint": {
    "checkpointId": "ckpt-ghi789",
    "agentId": "agent-abc123",
    "timestamp": "2026-01-25T10:30:00.000Z",
    "state": {
      "heldResources": ["resource-a"],
      "status": "active"
    },
    "sequenceNumber": 5
  }
}
```

롤백 실행

```
POST /api/recovery/rollback/:agentId
Content-Type: application/json

{
  "checkpointId": "ckpt-ghi789",
  "releaseResources": true
}
```

체크포인트 목록 조회

```
GET /api/recovery/checkpoints/:agentId
```

2.6 은행원 알고리즘

안전 상태 조회

```
GET /api/bankers
```

응답:

```
{
  "isSafe": true,
  "safeSequence": ["agent-1", "agent-3", "agent-2"],
  "availableResources": {
    "resource-a": 2,
    "resource-b": 1,
    "resource-c": 3
  },
  "analysis": {
    "totalAgents": 3,
    "totalResources": 3,
    "canFinishAll": true
  }
}
```

안전한 자원 요청

```
POST /api/bankers/request
Content-Type: application/json

{
  "agentId": "agent-1",
  "requests": {
```

```
    "resource-a": 1,
    "resource-b": 2
  }
}
```

2.7 그래프 상태 조회

```
GET /api/graph
```

응답:

```
{
  "agents": [
    {"id": "agent-1", "name": "Agent 1", "status": "active"},
    {"id": "agent-2", "name": "Agent 2", "status": "waiting"}
  ],
  "resources": [
    {"id": "resource-a", "name": "Resource A", "available": 0}
  ],
  "edges": [
    {"from": "agent-2", "to": "resource-a", "type": "waiting"}
  ],
  "cycles": []
}
```

---

3. 모니터링 대시보드

3.1 WebSocket 연결

```
const io = require('socket.io-client');
const socket = io('http://localhost:3003');

// 연결 성공
socket.on('connect', () => {
  console.log('Connected to Deadlock Detector');
});
```

3.2 이벤트 구독

그래프 업데이트 구독

```
// 구독
socket.emit('subscribe:graph');

// 이벤트 수신
socket.on('graph-updated', (data) => {
  console.log('Graph updated:', data);
  // {
  //   type: 'edge-added' | 'edge-removed' | 'agent-added' | ...
  //   payload: { ... }
  //   timestamp: "2026-01-25T10:30:00.000Z"
  // }
});
```

데드락 알림 구독

```
// 구독
socket.emit('subscribe:deadlock');

// 데드락 감지 알림
```

```
socket.on('deadlock-detected', (data) => {
  console.log('DEADLOCK DETECTED:', data);
  // {
  //   cycleId: "cycle-abc123",
  //   agents: ["agent-1", "agent-2"],
  //   detectedAt: "2026-01-25T10:30:00.000Z"
  // }
});

// 데드락 해결 알림
socket.on('deadlock-resolved', (data) => {
  console.log('Deadlock resolved:', data);
  // {
  //   cycleId: "cycle-abc123",
  //   victimId: "agent-2",
  //   resolvedAt: "2026-01-25T10:30:05.000Z"
  // }
});
```

에이전트 상태 변경 구독

```
socket.emit('subscribe:agents');

socket.on('agent-status-changed', (data) => {
  console.log('Agent status:', data);
  // {
  //   agentId: "agent-1",
  //   oldStatus: "active",
  //   newStatus: "waiting",
  //   timestamp: "2026-01-25T10:30:00.000Z"
  // }
});
```

3.3 시각화 예제 (React)

```
import React, { useEffect, useState } from 'react';
import io from 'socket.io-client';

function DeadlockMonitor() {
  const [graph, setGraph] = useState({ agents: [], edges: [] });
  const [deadlocks, setDeadlocks] = useState([]);

  useEffect(() => {
    const socket = io('http://localhost:3003');

    socket.emit('subscribe:graph');
    socket.emit('subscribe:deadlock');

    socket.on('graph-updated', (data) => {
      setGraph(data.payload);
    });

    socket.on('deadlock-detected', (data) => {
      setDeadlocks(prev => [...prev, data]);
    });

    return () => socket.disconnect();
  }, []);
```

```
  return (
    <div>
      <h1>Deadlock Monitor</h1>

      <h2>Agents ({graph.agents.length})</h2>
      <ul>
        {graph.agents.map(a => (
          <li key={a.id}>{a.name} - {a.status}</li>
        ))}
      </ul>

      <h2>Deadlocks ({deadlocks.length})</h2>
      {deadlocks.map(d => (
        <div key={d.cycleId} style={{color: 'red'}}>
          Cycle: {d.agents.join(' -> ')}
        </div>
      ))}
    </div>
  );
}
```

---

4. 트러블슈팅

4.1 자주 발생하는 문제

문제: 서버가 시작되지 않음

```
Error: Cannot find module './dist/index.js'
```

해결:

```
# TypeScript 빌드 실행
npm run build
```

문제: MongoDB 연결 실패

```
MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017
```

해결:

```
# MongoDB 상태 확인
brew services list | grep mongodb
# 또는
sudo systemctl status mongod

# MongoDB 시작
brew services start mongodb-community@7.0
# 또는
sudo systemctl start mongod
```

문제: Redis 연결 실패

```
Error: Redis connection to localhost:6379 failed
```

해결:

```
# Redis 상태 확인
redis-cli ping  # PONG 응답 확인

# Redis 시작
brew services start redis
# 또는
sudo systemctl start redis-server
```

문제: 데드락이 감지되지 않음

가능한 원인: 1. 그래프에 사이클이 없음 2. 탐지 간격이 너무 김 3. 엣지가 올바르게 추가되지 않음

해결:

```
# 그래프 상태 확인
curl http://localhost:3003/api/graph | jq

# 수동 탐지 실행
curl -X POST http://localhost:3003/api/deadlock/detect | jq
```

문제: 롤백 실패

```
Error: Checkpoint not found
```

해결:

```
# 체크포인트 목록 확인
curl http://localhost:3003/api/recovery/checkpoints/:agentId | jq

# 체크포인트가 없으면 먼저 생성
curl -X POST http://localhost:3003/api/recovery/checkpoint/:agentId
```

4.2 성능 최적화

대규모 그래프 최적화

```
# 환경 변수 조정
DETECTION_INTERVAL_MS=10000  # 탐지 간격 증가
MAX_CHECKPOINTS_PER_AGENT=5   # 체크포인트 수 감소
```

메모리 사용량 감소

```
# Node.js 메모리 제한 설정
NODE_OPTIONS="--max-old-space-size=512" npm start
```

4.3 로그 분석

```
# 로그 레벨 설정
LOG_LEVEL=debug npm run dev

# 로그 필터링
npm run dev 2>&1 | grep -E "(DEADLOCK|ERROR|WARN)"

# 로그 파일 저장
npm run dev 2>&1 | tee logs/app.log
```

4.4 건강 상태 확인

```
# 헬스 체크
curl http://localhost:3003/api/health

# 응답 예시
{
  "status": "ok",
  "timestamp": "2026-01-25T10:30:00.000Z",
  "services": {
    "mongodb": "connected",
    "redis": "connected"
  },
  "stats": {
    "uptime": 3600,
    "memoryUsage": "128MB",
    "agentCount": 10,
    "resourceCount": 5
  }
}
```

부록: API 에러 코드

코드	설명	해결 방법
AGENT_NOT_FOUND	에이전트 없음	에이전트 ID 확인
RESOURCE_NOT_FOUND	자원 없음	자원 ID 확인
RESOURCE_ALREADY_ALLOCATED	자원 이미 할당	해제 후 재요청
CHECKPOINT_NOT_FOUND	체크포인트 없음	체크포인트 생성
CYCLE_NOT_FOUND	사이클 없음	탐지 재실행
UNSAFE_STATE	안전하지 않은 상태	자원 해제 후 재시도
VALIDATION_ERROR	입력 검증 실패	요청 파라미터 확인

문서 버전: 1.0.0 최종 수정: 2026-01-25 작성자: 홍익대학교 컴퓨터공학과 졸업 프로젝트 팀